ConstructionIQ — System Architecture & Database Schema

1. Architecture overview

ConstructionIQ follows a **layered, service-oriented architecture** on top of a single relational database. It is not microservices — for a project of this scope, a modular monolith is easier to build, reason about, and demo, while still being organized so pieces *could* be split out later (especially the AI layer).

Realtime: Socket.IO pushes events (low-stock alert, new incident, delivery delay) from the service layer to subscribed clients.

Deployment: Vercel (frontend) · Render/Railway/Fly.io (backend) · Neon or Supabase (PostgreSQL)

Why this shape

- **API layer stays thin.** Controllers only authenticate, validate, and call services. This keeps business rules (e.g. "a material request needs PM approval above ₹50,000") testable and out of the HTTP layer.
- **AI service layer is isolated** because its calls are slow, non-deterministic, and billed per token. It's the one place designed for async/background execution (a queue such as BullMQ is a natural addition once daily-report generation runs at scale).
- **RBAC is enforced at two levels**: route-level middleware (does this role even have access to this endpoint) and row-level checks in services (can this specific Site Engineer touch *this* project — i.e. are they on its `PROJECT_TEAM`).
- **Single Postgres database** keeps joins across modules (e.g. "budget vs. actual material cost") cheap. Splitting into per-module databases would only pay off at a scale this project isn't targeting.

Request flow example — material request approval

1. Site Engineer submits a request → `POST /projects/:id/material-requests`.
2. API layer authenticates JWT, confirms role is `site_engineer` and that the user is on the project team.
3. Core service layer calls the **duplicate-request detection** AI service before saving, to warn if a similar request already exists.
4. Request is saved with `status = pending`.
5. Socket.IO emits a `material_request.created` event to the assigned Project Manager.
6. PM approves → service layer updates `status`, decrements nothing yet (decrement happens on actual issue, tracked in `MATERIAL_TRANSACTION`).

---

2. Database schema

Relational design in PostgreSQL, managed through Prisma. All primary keys are UUIDs. Timestamps (`created_at`, `updated_at`) are included on every table but omitted below for brevity except where they carry business meaning.

2.1 Identity & access

`users`

Column	Type	No
id	uuid PK	
name	varchar	
email	varchar, unique	
password_hash	varchar	
role	enum(`admin`,`project_manager`,`site_engineer`,`contractor`,`supplier`)	
phone	varchar	nu
is_active	boolean	de tru
created_at / updated_at	timestamp	

`project_team` (join table — who's assigned to which project, and in what capacity)

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
user_id	uuid FK → users.id	
role_on_project	varchar	e.g. "lead engineer"
unique(project_id, user_id)		

2.2 Projects

`projects`

Column	Type	Notes
id	uuid PK	
name	varchar	
description	text	nullable
location	varchar	
latitude / longitude	decimal	nullable, for map view
start_date / end_date	date	
status	enum(`planning`, `active`, `delayed`, `on_hold`, `completed`)	
budget_estimated	decimal(14,2)	
manager_id	uuid FK → users.id	
created_at / updated_at	timestamp	

`project_milestones`

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
title	varchar	
target_date	date	
completed_date	date	nullable
status	enum(`pending`, `in_progress`, `completed`, `delayed`)	

2.3 Materials & inventory

`materials` (catalog — one row per material type, shared across projects)

Column	Type	Notes
id	uuid PK	
name	varchar	e.g. "Cement (OPC 53)"
category	varchar	
unit	varchar	bag, kg, ton, piece...
unit_cost	decimal(10,2)	

`material_inventory` (stock level per project + material)

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
material_id	uuid FK → materials.id	
quantity_available	decimal(12,2)	
warehouse_location	varchar	nullable
low_stock_threshold	decimal(12,2)	
unique(project_id, material_id)		

`material_requests`

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
material_id	uuid FK → materials.id	
requested_by	uuid FK → users.id	
quantity_requested	decimal(12,2)	
status	enum(`pending`, `approved`, `rejected`, `fulfilled`)	
approved_by	uuid FK → users.id	nullable
ai_duplicate_flag	boolean	set by AI duplicate-detection
created_at	timestamp	

`material_transactions` (append-only ledger of actual stock movement)

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
material_id	uuid FK → materials.id	
type	enum(`received`, `issued`, `returned`)	
quantity	decimal(12,2)	
reference_id	uuid	nullable, links to material_request or delivery
created_at	timestamp	

2.4 Equipment

`equipment`

Column	Type	Notes
id	uuid PK	
name	varchar	
type	varchar	excavator, crane, mixer...
status	enum(`available`, `in_use`, `under_maintenance`)	
purchase_date	date	nullable

`equipment_bookings`

Column	Type	Notes
id	uuid PK	
equipment_id	uuid FK → equipment.id	
project_id	uuid FK → projects.id	
booked_by	uuid FK → users.id	
start_time / end_time	timestamp	
status	enum(`booked`, `in_progress`, `completed`, `cancelled`)	

`equipment_usage_logs`

Column	Type	Notes
id	uuid PK	
equipment_id	uuid FK → equipment.id	
project_id	uuid FK → projects.id	
date	date	
hours_used	decimal(5,2)	
fuel_used_liters	decimal(6,2)	nullable

2.5 Workforce

`workers`

Column	Type	Notes
id	uuid PK	
name	varchar	
contractor_id	uuid FK → users.id	the contractor supervising them
role	varchar	mason, electrician...
contact	varchar	nullable

`attendance`

Column	Type	Notes
id	uuid PK	
worker_id	uuid FK → workers.id	
project_id	uuid FK → projects.id	
date	date	
status	enum(`present`,`absent`,`half_day`,`leave`)	
shift	varchar	nullable
overtime_hours	decimal(4,2)	default 0
unique(worker_id, project_id, date)		

2.6 Deliveries & suppliers

`deliveries`

Column	Type	Note
id	uuid PK	
project_id	uuid FK → projects.id	
supplier_id	uuid FK → users.id	role =
material_id	uuid FK → materials.id	
quantity	decimal(12,2)	
scheduled_date	date	
actual_date	date	nulla
status	enum(`scheduled`,`in_transit`,`delivered`,`delayed`,`cancelled`)	
invoice_url	varchar	nulla store S3/C
proof_of_delivery_url	varchar	nulla

2.7 Budget & expenses

`budgets` (planned budget lines per category)

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
category	enum(`material`,`equipment`,`labor`,`other`)	
estimated_amount	decimal(14,2)	

`expenses` (actual spend, logged over time)

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
category	enum(`material`, `equipment`, `labor`, `other`)	
amount	decimal(14,2)	
description	varchar	nullable
date	date	
created_by	uuid FK → users.id	
ai_flagged_unusual	boolean	set by smart cost analysis

2.8 Incidents & daily reports

`incidents`

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
reported_by	uuid FK → users.id	
type	enum(`accident`,`equipment_failure`,`safety_violation`,`other`)	
severity	enum(`low`,`medium`,`high`,`critical`)	
description	text	
image_urls	text[]	array of S3/Cloudi URLs
status	enum(`open`,`investigating`,`resolved`)	
resolution_notes	text	nullable
created_at	timestamp	

`daily_site_reports`

Column	Type	Notes
id	uuid PK	
project_id	uuid FK → projects.id	
site_engineer_id	uuid FK → users.id	
date	date	
work_completed	text	
workforce_count	integer	
weather	varchar	nullable
material_usage_notes	text	nullable
equipment_usage_notes	text	nullable
delay_notes	text	nullable
image_urls	text[]	
ai_generated_summary	text	nullable, output of AI daily-report feature
unique(project_id, date)		

2.9 AI & notifications

`ai_insights` (stores generated predictions/insights for auditability)

Column	Type
id	uuid PK
project_id	uuid FK → projects.id
type	enum(`delay_prediction`,`cost_anomaly`,`duplicate_request`,`assistant_qu
content	text
confidence_score	decimal(4,3)
created_at	timestamp

`notifications`

Column	Type	Notes
id	uuid PK	
user_id	uuid FK → users.id	
message	varchar	
type	varchar	e.g. "low_stock", "incident", "approval_needed"
read	boolean	default false
created_at	timestamp	

3. Key relationships summary

- `users` (manager) **1—N** `projects`
- `users` **M—N** `projects` via `project_team` (engineers, contractors, PMs assigned)
- `projects` **1—N** `material_requests`, `material_inventory`, `equipment_bookings`, `attendance`, `deliveries`, `budgets`, `expenses`,

`incidents`, `daily_site_reports`

- `materials` **1—N** `material_requests`, `material_inventory`, `material_transactions`, `deliveries`
- `equipment` **1—N** `equipment_bookings`, `equipment_usage_logs`
- `workers` **1—N** `attendance`; `users` (contractor) **1—N** `workers`
- `users` (supplier) **1—N** `deliveries`

4. Notes on scaling this design

- **Row-level access control:** since RBAC alone (role-level) isn't enough — a Site Engineer should only see *their* project's data — every service-layer query should filter by `project_team` membership, not just role.
- **Audit trail:** `material_transactions` and `ai_insights` are intentionally append-only logs rather than mutable fields, so history (and AI explainability) survives edits.
- **AI cost control:** cache `ai_insights` per project/day so the delay-prediction model isn't re-run on every dashboard load — regenerate on a schedule or on data change, not on every request.